



Real-time defect monitoring in material extrusion 3D printing using optimized YOLO models

Abdul Rahman Sani¹ · Abbas Z. Kouzani¹ · Ali Zolfagharian¹

Received: 27 July 2025 / Accepted: 15 October 2025 / Published online: 12 November 2025
© The Author(s), under exclusive licence to Springer Nature Switzerland AG 2025

Abstract

Real-time defect detection is vital in Fused Deposition Modeling (FDM) for ensuring the quality of 3D-printed objects and enabling autonomous control systems that can reduce material waste, improve reliability, and minimize human intervention. This work presents the first applied demonstration of a real-time dual-camera defect-detection system on a low-cost embedded platform for multi-angle defect detection during active 3D printing. A custom dataset of four common extrusion defects: stringing, spaghetti, under-extrusion, and over-extrusion, was developed and used to fine-tune lightweight YOLOv11n and YOLOv12n models through transfer learning. The models were further optimized via multi-objective hyperparameter tuning using the Optuna framework, adapted for low-power embedded inference to balance accuracy, localization precision, and speed with dual cameras for multi-angle monitoring. The results demonstrated that the optimized models jointly improve detection accuracy and localization precision while maintaining real-time performance. The system reliably identifies all four extrusion-related defects and validated a perception-to-action pipeline linking camera inference to printer-firmware response, confirming closed-loop feasibility. These findings establish a practical foundation for AI-augmented additive manufacturing and paves the way toward autonomous, closed-loop 3D and 4D printing systems.

Keywords Real-time monitoring · Defect detection · Additive manufacturing · Artificial intelligence · AI2AM

1 Introduction

Additive manufacturing (AM), also known as 3D printing, has significantly transformed modern manufacturing by enabling the layer-by-layer production of physical components directly from their computer-aided design (CAD) models. 3D printing enables the production of complex objects with customized designs that are often difficult to create using traditional manufacturing techniques. Among the different AM methods, FDM is commonly used because of its low cost and user-friendly operation with a wide range of filaments. In FDM 3D printing, thermoplastic filament is extruded through a heated nozzle and deposited on the

print bed to fabricate the desired components from digital models. Despite these advantages, FDM 3D printing and 4D printing remain vulnerable to extrusion defects. These defects not only compromise the mechanical integrity and surface quality of printed parts but can also result in material waste and extended production times [1].

Common extrusion defects include stringing, spaghetti, under-extrusion, and over-extrusion [2]. Each of these defects occurs due to distinct reasons during material deposition [3]. Stringing is characterized by fine, unwanted threads of filament formed when the nozzle travels between parts of a print without sufficient retraction settings. Spaghetti is a severe extrusion failure that occurs when a print loses adhesion to the print bed, and the filament extrudes into the air uncontrollably. Over-extrusion occurs when excess filament is deposited, often leading to bulging on the surface of a 3D-printed component. On the other hand, under-extrusion is caused by insufficient filament extrusion, which can weaken the structural bond of the component or leave gaps between layers. These defects compromise the dimensional accuracy, mechanical properties, and surface finish of printed components, highlighting the need for

✉ Ali Zolfagharian
a.zolfagharian@deakin.edu.au
Abdul Rahman Sani
s221149018@deakin.edu.au
Abbas Z. Kouzani
abbas.kouzani@deakin.edu.au

¹ Deakin University, Geelong, Australia

robust quality monitoring mechanisms for FDM 3D printing operations [3–5].

Traditional quality monitoring methods, such as visual inspection of 3D-printed components when printing is completed, or manual inspection during the active printing, is not feasible for real-time defect mitigation and is not practical in an automated production environment. Recent advances in artificial intelligence (AI) and computer vision, particularly deep learning, have enabled the real-time detection of these defects through analysis of image or video data during active printing [6]. For instance, Khusheef et al. demonstrated the effectiveness of LSTM-based temporal models for real-time process monitoring in FDM, highlighting the potential of deep learning for dynamic fault detection [7]. Real-time defect detection is a key factor in reducing waste, reducing production costs, and ensuring 3D-printed component quality in FDM 3D Printing [7]. For this purpose, the You Only Look Once (YOLO) family of single-stage object detectors stands out for offering high-speed, high-accuracy performance suitable for real-time defect detection applications [8].

While earlier YOLO models like YOLOv5 and YOLOv8 have demonstrated strong performance in various applications, they often struggle to detect subtle or small-scale 3D printing defects such as stringing, particularly under variable lighting or geometric conditions. Additionally, deploying these YOLO models on low-resource edge devices such as the Raspberry Pi with multiple camera views is constrained due to limited computational resources [9]. Recently, two new architectures have been proposed in the YOLO family: YOLOv11 and YOLOv12. These new YOLO versions bring significant improvements, including anchor-free detection heads, transformer-based attention modules, improved cross-stage partial connections, and an enhanced feature fusion network [10, 11]. These advancements are designed to enhance sensitivity to small-scale features while maintaining computational efficiency suitable for real-time deployment on embedded systems [12].

Even with these advanced YOLO architectures, recent research in the literature mainly uses standard (fine-tuned) hyperparameters, which could prevent models from reaching their full performance potential on the data related to real-time defects in FDM 3D printing. Additionally, there remains a lack of comprehensive comparative evaluations between YOLOv11 and YOLOv12 architectures within the context of AM defect detection. To address these limitations, in this study, we implement Optuna, a state-of-the-art hyperparameter optimization (HPO) framework capable of performing multi-objective optimization. This allows us to jointly maximize both $\text{mAP}@0.5$ and $\text{mAP}@0.5:0.95$, which are critical metrics for evaluating object localization precision and detection accuracy across varying

intersection-over-union (IoU) thresholds [13]. Moreover, we leverage transfer learning by initializing YOLOv11n and YOLOv12n with pre-trained weights from the COCO dataset, thus accelerating convergence and enhancing generalization on our comparatively small dataset of FDM printing defects [14]. This fine-tuning process ensures that the models adapt effectively to subtle, real-world defect patterns that are not present in generic datasets.

Although physics-based models can capture extrusion dynamics, they are not practical for real-time embedded deployment, where response latency and computational constraints dominate. In contrast, Optuna offers a data-driven, multi-objective optimization framework that efficiently explores high-dimensional parameter spaces using Bayesian sampling and pruning strategies [13]. This enables systematic balancing between accuracy, localization precision, and speed, the key objectives for embedded vision tasks in FDM 3D printing. The choice of Optuna was therefore motivated by its suitability for resource-constrained, adaptive optimization rather than physical process modeling, aligning with recent progress in AM studies that emphasize data-driven process optimization in intelligent AM systems [15, 16].

Optuna enables efficient exploration of large hyperparameter spaces, systematically balancing detection accuracy and localization precision while avoiding excessive computational cost. This makes it particularly well-suited for embedded applications, such as Raspberry Pi based defect detection in AM. This work builds on our prior work [3], we conducted a comprehensive benchmark of five YOLO models (YOLOv5, YOLOv8, YOLOv9, YOLOv10, and YOLOv11), including their variants on a custom 3D printing defect dataset. While that study established baseline performance across versions, it was limited to offline inference and manual threshold tuning. In contrast, the present work focuses on real-time defect detection using lightweight YOLOv11n and YOLOv12n models, optimized via multi-objective hyperparameter tuning using Optuna. Additionally, this paper implements a full embedded deployment on an embedded system with dual-camera input for Industry 4.0-compatible, resource-efficient monitoring.

Hence, building on these advancements, this study presents an applied, real-time defect-detection pipeline for FDM 3D printing, deploying both YOLOv11n and YOLOv12n models on a Raspberry Pi 5B embedded platform. A custom dataset containing four common extrusion-related defects was collected from ROBOFLOW Universe, then annotated and curated using the ROBOFLOW platform to ensure consistency and reproducibility. Optuna was employed to perform multi-objective HPO, jointly maximizing $\text{mAP}@0.5$ and $\text{mAP}@0.5:0.95$, while maintaining inference efficiency suitable for embedded deployment.

To the best of our knowledge, this is the first applied study to implement a real-time dual-camera defect-detection system using YOLOv11n and YOLOv12n on a low-cost embedded Raspberry Pi 5B platform for FDM 3D printing. While previous research focused on earlier YOLO models or using default configurations, our approach integrates optimized YOLO models, deployment on a low-cost embedded system and real-time monitoring in a unified pipeline. Our system captures dual-angle video feeds using two Raspberry Pi Camera Module 3 units and performs lightweight, real-time inference using an optimized YOLOv11n and YOLOv12n models adapted for extrusion defect detection. The key contributions of this study are as follows:

- Implementing and performance metrics analysis of YOLOv11n and YOLOv12n models on the FDM 3D printing defect dataset.
- Multi-objective hyperparameter optimization using Optuna to improve detection accuracy while maintaining deployment efficiency.
- Real-time deployment and benchmarking of optimized models on a Raspberry Pi 5B using two Raspberry Pi Camera Module 3 units in a dual-view configuration.

Beyond integrating YOLO models on embedded hardware, this work establishes a unified, application-driven framework for real-time extrusion-defect detection under resource-constrained conditions. Specifically, the combination of Optuna-based multi-objective optimization with a custom dataset provides a systematic way to balance detection accuracy, localization precision, and inference speed for AM. Unlike previous research works that focused on single metrics or offline evaluation, our approach explicitly addresses trade-offs relevant to embedded deployment and robustness in real printing environments.

By demonstrating practical feasibility on a Raspberry Pi 5B, this work bridges the gap between academic benchmarking and industrial implementation. The validated perception-to-action pipeline further establishes the groundwork for closed-loop, defect-adaptive 3D printing where real-time monitoring can directly inform corrective actions, advancing toward autonomous and adaptive manufacturing systems [17]. This work will contribute to the development of scalable, AI-augmented solutions align with the AM industry 4.0.

The remainder of this paper is organized as follows: Sect. 2 reviews related work on defect detection in AM. Section 3 presents the methodology, beginning with dataset development, YOLOv11n and YOLOv12n architectures, transfer learning and fine-tuning, Optuna-based multi-objective hyperparameter optimization, the training setup, and deployment on embedded system. Section 4 reports the

results, including overall detection performance, defect-wise analysis, and embedded deployment outcomes. Section 5 discusses the findings, practical implications, and limitations, and includes analyzing closed-loop system feasibility and latency. Finally, Sect. 6 concludes this work, summarizing the contributions and outlining future work toward closed-loop, AI2AM.

2 Related work

Recent advancements in computer vision have significantly enhanced real-time monitoring in AM, particularly for FDM 3D printing. The YOLO object detection framework has emerged as a leading framework due to its balance between accuracy and speed, which makes it useful in areas where you want precision and real-time inference at the same time. A comprehensive review by Siddiqui et al. highlighted that condition-based monitoring has become a cornerstone of intelligent additive manufacturing, emphasizing the growing role of real-time, data-driven approaches such as vision- and AI-based sensing in achieving autonomous quality assurance [18]. Early YOLO models, such as YOLOv3, YOLOv5, and YOLOv8, demonstrated sufficient performance in basic defect detection for FDM printing by identifying common defects like layer shifting and warping [19]. However, these models are not able to identify small and specific extrusion-related defects such as stringing, spaghetti, under-extrusion, and over-extrusion.

The development of more advanced YOLO models such as YOLOv8, YOLOv9, YOLOv10, and YOLOv11 has enabled improved performance in detecting complex defect types [3, 6, 20]. YOLOv8 has been widely adopted due to its anchor-free architecture, improved backbone, and real-time speed. Recently, Nikam et al. [21] applied YOLO to defect detection in L-DED processes in the metal AM domain, demonstrating that lightweight detection models can detect flash, voids, and roughness under real-time constraints. Cao et al. [9] demonstrated the capability of a YOLOv8-based model to detect 3D printing defects with mAP scores exceeding 90%, while an enhanced YOLOv8 in another study achieved even higher precision with real-time inference speeds [22]. Shabani et al. [23] further integrated YOLO with extrusion width monitoring systems, achieving dimensional accuracy of up to 90 μm , thereby demonstrating the feasibility of deploying YOLO models in precision-critical AM workflows.

Building on the architectural foundations laid by earlier YOLO models, the introduction of YOLOv11 marked a significant advancement in object detection for manufacturing applications. YOLOv11 integrates multiple architectural innovations, including anchor-free detection heads,

transformer-based attention modules, and a Feature Fusion Pyramid (FFP) network [24]. These modifications enhance the model's ability to detect subtle and low-contrast anomalies with minimal computational overhead [25]. Building upon this, YOLOv12 introduces dynamic stride alignment, decoupled detection heads, and enhanced feature-level fusion strategies [26, 27]. Although still relatively new to the AM domain, YOLOv12 has demonstrated superior performance in small object detection and multi-scale localization in other fields, making it a promising candidate for FDM defect detection [28].

Despite these advances, existing studies rely on manually tuned or default hyperparameters, which may result in suboptimal model performance. HPO frameworks such as Optuna are well-established for fine-tuning deep learning models using Bayesian optimization and efficient pruning. While these methods are widely used in computer-vision research, their applied adaptation to embedded, real-time AM defect detection tasks remain scarce. Akiba et al. [13] introduced Optuna as a lightweight, scalable HPO solution capable of navigating complex search spaces for both single and multi-objective optimization problems.

In prior work, most studies either used fixed learning rates or explored only a small subset of the parameter space. Our previous study [3] explored confidence threshold and anchor size tuning but did not employ a full HPO pipeline. However, recent research in other domains demonstrates the effectiveness of structured HPO. For example, Ramos et al. [29] performed a two-phase HPO strategy for YOLOv8 in wildfire detection using One-Factor-At-a-Time (OFAT) analysis, followed by random search. Their optimized model achieved a 5.09% improvement in mAP@0.5:0.95, highlighting the significant benefits of systematic tuning for real-time safety-critical tasks.

Recent studies have explored the use of lightweight YOLO models such as YOLOv5 and YOLOv8 on embedded platforms like Raspberry Pi and Jetson Nano, but systematic evaluations of more advanced models like YOLOv11 and YOLOv12 remain limited. For example, Ghatte et al. [30] compared YOLOv8 and YOLOv11 for agricultural grading of arecanuts, finding that YOLOv8-nano achieved higher accuracy (98.25%) and faster inference using grid search for hyperparameter tuning, while YOLOv11's performance suffered due to feature overlap, indicating that architecture alone is insufficient without proper optimization. Transfer learning has also proven effective in overcoming small dataset limitations, with Yosinski et al. [31] and Li et al. [32] showing that initializing models with pre-trained weights (e.g., from COCO) improved convergence and generalization. Hendrycks et al. [33] further showed that pretraining improves robustness and reduces uncertainty, which is critical in real-time monitoring applications.

While previous work has shown that lightweight YOLO models like YOLOv5 and YOLOv8 can be deployed on edge devices such as Raspberry Pi and Jetson Nano, there remains a lack of systematic evaluation of more advanced models like YOLOv11 and YOLOv12 for embedded real-time defect detection. Given the rise of digital twins and demands for closed-loop 3D printing, deploying optimized high-performance models remains an underexplored but essential research direction.

Recent advances have highlighted the role of real-time monitoring in improving AM process reliability and defect detection. Yue et al. [6] proposed an in-situ detection system for continuous fibre-reinforced composites, offering insight into vision-based anomaly identification during the printing process. Shabani et al. [23] focused on polymer-based FDM and developed a system to measure extrusion width using machine learning, enabling immediate response to dimensional inconsistencies. In another study, authors have also explored embedded-friendly models, such as a YOLOv8-based lightweight detector, achieving 97.5% mAP in real-time FDM monitoring [34]. Similarly, Ajay and Shrivastava developed a single-camera vision system for real-time bead-geometry measurement in WAAM, underscoring the feasibility of vision-based dimensional monitoring in additive manufacturing [35]. However, most studies lack deployment on constrained embedded systems with dual-camera setups. This paper addresses this limitation through real-time implementation on a Raspberry Pi 5B using YOLOv11 and YOLOv12 models.

In summary, the current literature demonstrates the effectiveness of YOLO-based detectors in defect monitoring across AM and adjacent domains. However, critical research gaps remain, including:

- A lack of comparative evaluation between YOLOv11 and YOLOv12 in the context of 3D printing defects.
- Limited exploration of multi-objective hyperparameter optimization using frameworks like Optuna for FDM defects.
- Sparse use of transfer learning specifically tailored for extrusion-related defects.
- Minimal evidence of real-time embedded deployment of advanced YOLO models for FDM 3D printing.

To bridge these gaps, this study conducts an applied comparative evaluation of YOLOv11n and YOLOv12n for real-time extrusion-defect detection. The models are fine-tuned via transfer learning, optimized using Optuna for multi-objective performance, and deployed on a dual-camera Raspberry Pi 5B platform to validate their practical feasibility in embedded FDM 3D printing environments.

Table 1 Statistical summary of the FDM defect dataset used for model training, validation, and testing

Defect Class	Training	Validation	Testing	Total
Under-extrusion	651	146	152	949
Over-extrusion	686	153	142	981
Stringing	648	142	127	917
Spaghetti	1,006	193	222	1,421
Total	2,991	634	643	≈ 4,267

3 Methodology

3.1 Dataset collection and preparation

To ensure reliable and generalizable model performance in real-time 3D printing environments, a custom dataset was assembled and adapted from publicly available sources on the Roboflow Universe repository [36]. This dataset was specifically curated to represent four extrusion-related defect categories that frequently occur in FDM 3D printing. Each image was manually annotated using bounding boxes to accurately mark the defect regions, providing high-quality ground truth for supervised training.

To increase data diversity and reduce the risk of over-fitting, several augmentation techniques were applied to the training dataset using the Roboflow platform. These included random rotations within $\pm 15^\circ$, horizontal and vertical shearing up to 10%, and brightness variations of up to 15%. These augmentations were used to simulate camera motion, print bed vibration, and ambient lighting variations, which are common during real-time print monitoring. Automatic image resizing and alignment were also applied to reduce geometric distortions and maintain consistent input dimensions.

The dataset used in this study consists of 4,267 labeled images, categorized into four extrusion-related defect types: 949 images for under-extrusion, 981 images for over-extrusion, 917 images for stringing, and 1,421 images for spaghetti. The dataset was divided into training, validation, and test sets using a stratified split to ensure a balanced

representation across all defect classes. Specifically, approximately 70% of the data was allocated to training, 15% to validation, and 15% to testing. This resulted in 2,990 training images, 634 validation images, and 643 test images. The class-wise distribution across these subsets is summarized in Table 1. Each image was resized to 640×640 pixels to meet YOLO input requirements while preserving aspect ratio to avoid geometric distortion. The training dataset was further expanded to 8,970 images after augmentation. Figure 1 displays sample images from the datasets, illustrating the intra-class variability and visual complexity of the four targeted FDM extrusion defect types.

3.2 YOLOv11 and YOLOv12 architecture

In this study, two recent object detection models, YOLOv11 and YOLOv12, are evaluated for their effectiveness in identifying extrusion-related defects in FDM 3D printing. Both models build upon the anchor-free detection paradigm introduced in YOLOv8, with substantial enhancements to feature extraction, attention mechanisms, and computational efficiency. YOLOv11 expands the YOLOv8 framework with targeted architectural improvements aimed at boosting detection accuracy while maintaining lightweight characteristics suitable for edge deployment. It introduces the C3K2 module, a modified Cross Stage Partial (CSP) block with a kernel size of 2, which facilitates efficient yet powerful feature extraction. Additionally, it integrates the Spatial Pyramid Pooling – Fast (SPPF) module to enhance multi-scale context aggregation, improving the model's robustness to variable object sizes and shapes. In practical terms, the C3K2 module reduces computational load while preserving strong feature extraction, making the network lightweight for embedded devices. Similarly, SPPF aggregates multi-scale features efficiently, allowing the model to detect both large extrusion errors (e.g., spaghetti) and fine defects (e.g., under-extrusion) without excessive computational cost.

A key innovation in YOLOv11 is the Convolutional block with Parallel Spatial Attention (C2PSA) module,

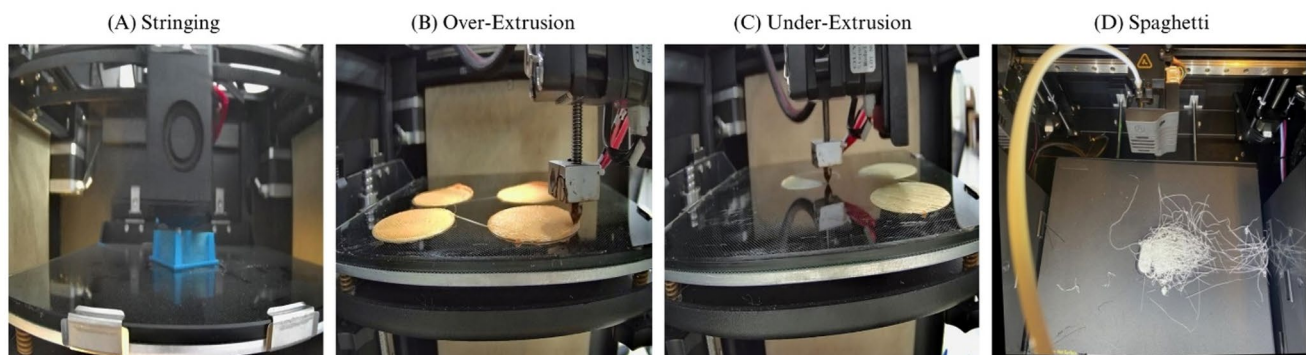


Fig. 1 Sample images illustrating the four extrusion-related defects: **A** Stringing, **B** Over-Extrusion, **C** Under-extrusion, and **D** Spaghetti

which applies spatial attention mechanisms across parallel convolutional paths. The value of C2PSA lies in its ability to direct the model's attention toward critical print regions, such as thin filament strands, while suppressing background noise, thereby improving defect detection reliability in real printing scenarios. This facilitates the network to focus on salient spatial regions within the feature map, improving detection of small or overlapping defects such as stringing and clustered extrusion errors. These architectural elements collectively improve accuracy while preserving high-speed inference, critical for real-time deployment on embedded platforms. A high-level overview of the YOLOv11 architecture is illustrated in Fig. 2.

Another key enhancement is the integration of Flash Attention, which significantly reduces memory access overhead while accelerating attention computation, resulting in faster inference with minimal trade-offs in accuracy. Practically, Flash Attention makes training and inference faster and less memory intensive, enabling deployment on hardware-constrained platforms like the Raspberry Pi without sacrificing accuracy. YOLOv12 also adopts 7×7 separable

convolutions, increasing the receptive field while maintaining a low parameter count. Furthermore, it removes positional encodings typically found in transformer-based heads to streamline inference and reduce latency. These design choices strike a balance between expanding the receptive field (to detect large-scale extrusion anomalies) and minimizing latency, which is critical for real-time defect detection in AM.

Together, these innovations make YOLOv12 exceptionally well-suited for real-time detection of subtle and complex extrusion defects under the computational constraints of embedded hardware. The full architectural overview of YOLOv12 is presented in Fig. 3.

3.3 Transfer learning

To accelerate convergence and improve generalization on the FDM 3D printing defect dataset, transfer learning was utilized for both YOLOv11n and YOLOv12n models. The models were initialized with pre-trained weights from the Microsoft COCO dataset [38], which contains over 200,000

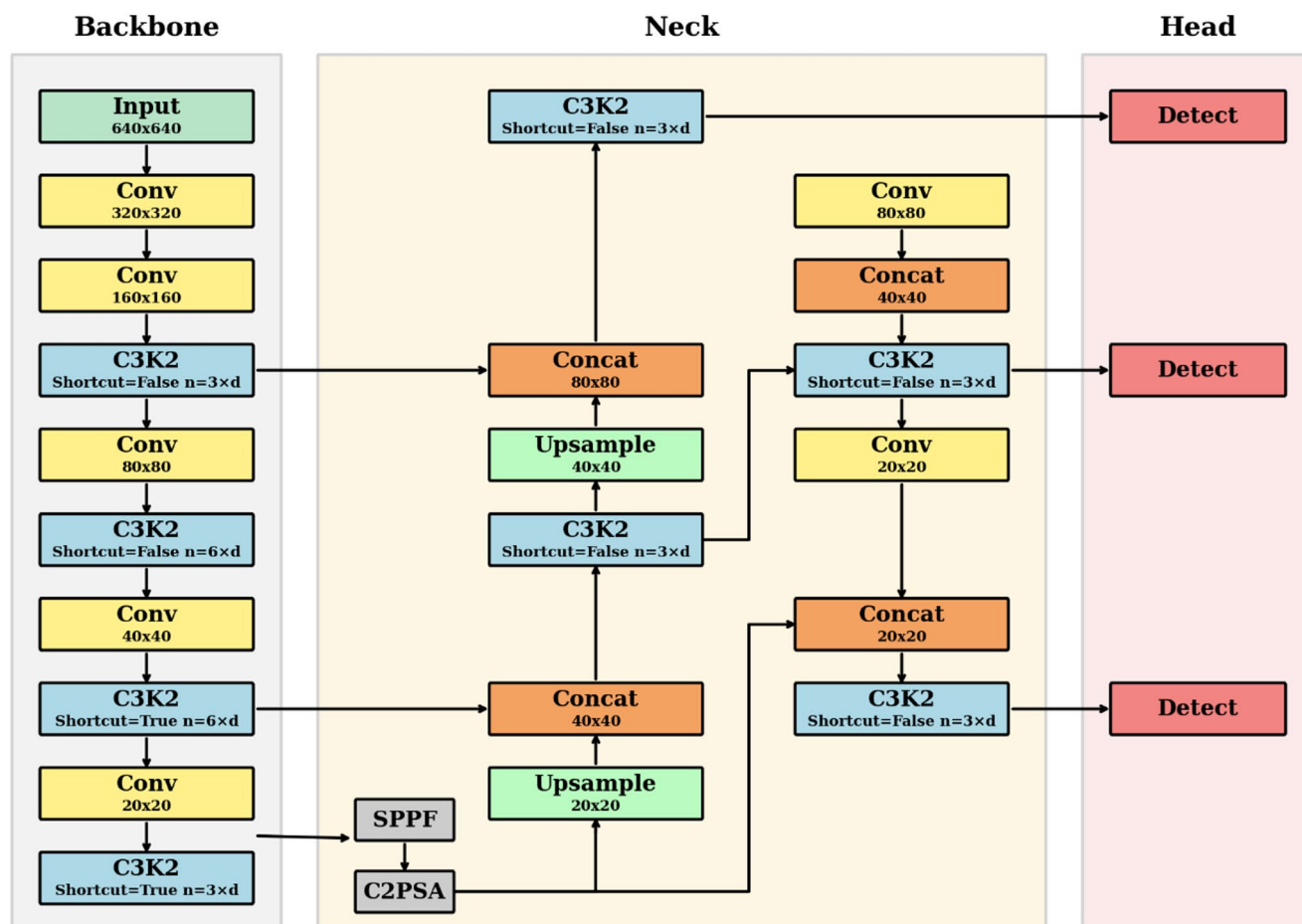


Fig. 2 YOLOv11 architecture featuring lightweight modules like C3K2, SPPF, and C2PSA to improve spatial attention and efficiency in detecting small-scale FDM defects (Adapted from [37] Copyright 2024 by the authors, licensed under CC BY 4.0, MDPI, Basel, Switzerland)

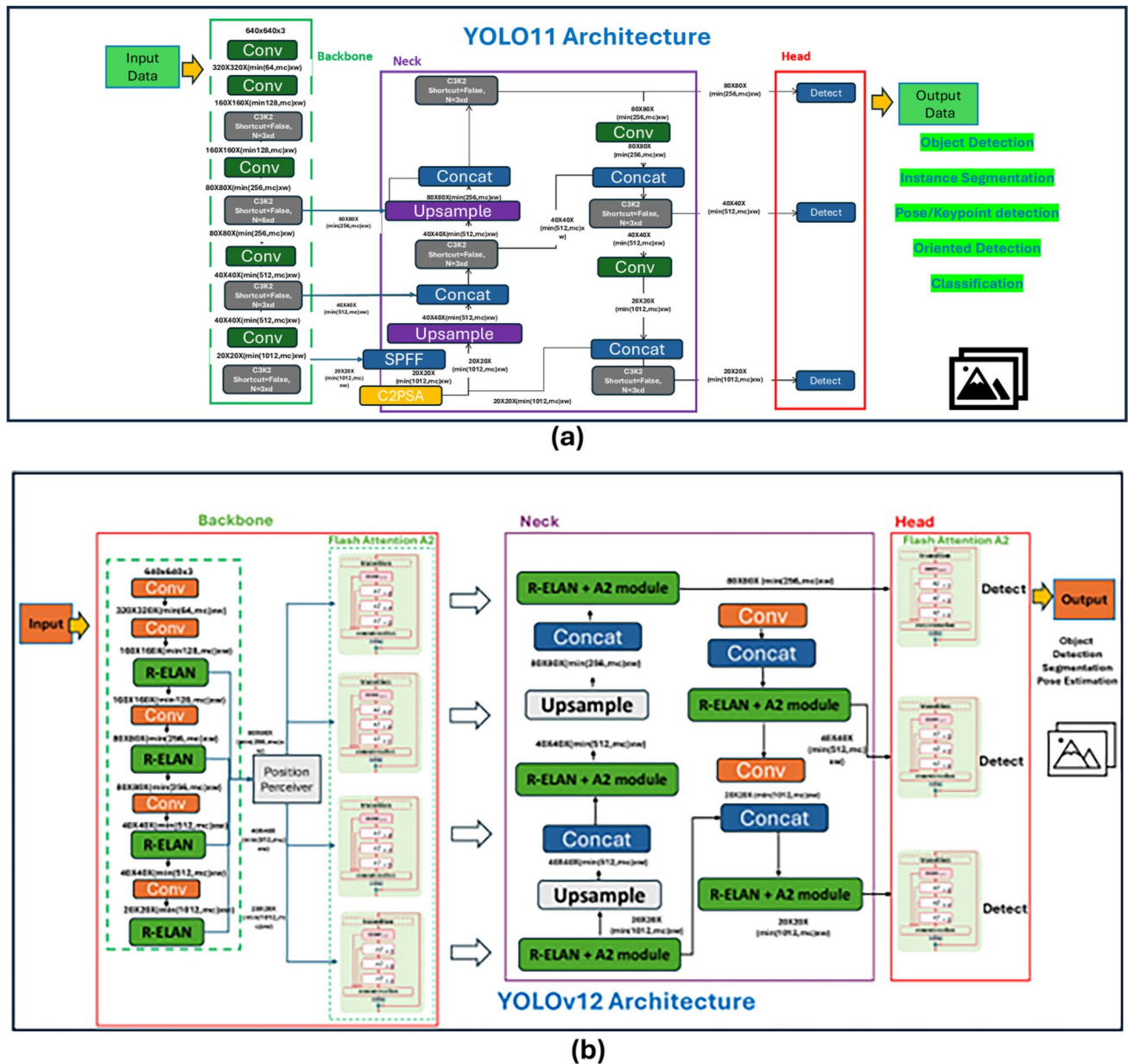


Fig. 3 YOLOv12 architecture integrating R-ELAN, Area Attention, Flash Attention, and separable convolutions to achieve high precision and real-time inference in industrial 3D printing applications (Repro-

duced with permission from [24] Copyright 2025 under CC BY 4.0, *Artificial Intelligence Review*, Springer Nature)

labelled images spanning 80 object categories. These pre-trained weights provide a foundation of general-purpose visual features, enabling faster and more effective adaptation to the FDM defect detection task. All training and fine-tuning tasks were performed on Google Colab Pro using an A100 GPU environment.

During training, the initial layers of the backbone were frozen to retain low-level features such as edges, textures, and contours, while the deeper layers were fine-tuned to specialize in detecting extrusion-related defects like stringing, spaghetti, under-extrusion, and over-extrusion. This

approach reduced the risk of overfitting, which is particularly important given the limited size of the custom dataset. Transfer learning also contributed to training efficiency, enabling high validation performance in fewer epochs compared to training from scratch. Fine-tuning was conducted using the Ultralytics YOLO framework, which supports custom class integration and pre-trained model loading. The complete process is outlined in Table 2, which summarizes the transfer learning pipeline adopted for both YOLOv11n and YOLOv12n.

Table 2 Transfer learning procedure for adapting YOLOv11n and YOLOv12n models to the FDM extrusion defect detection task

Algorithm 1: Transfer Learning for YOLOv11n and YOLOv12n

Input: COCO-pre-trained YOLO model, custom-labelled FDM defect dataset**Output:** Fine-tuned YOLO model for extrusion defect detection

1. Load COCO-pretrained weights for YOLOv11n or YOLOv12n
2. Replace the classification head with one suited for four extrusion defect classes
3. Freeze early layers of the backbone to retain general visual features
4. Apply dataset-specific data augmentation and preprocessing
5. Use the SGD optimizer with a cosine annealing scheduler for initial training
6. Monitor validation performance (mAP@0.5) during training
7. Gradually unfreeze additional layers based on convergence behaviour
8. Save the best-performing model weights for evaluation and deployment

By using transfer learning, both models were able to leverage pre-learned, general-purpose features while effectively adapting to the distinct visual characteristics of extrusion defects commonly observed in FDM printing.

3.4 Multi-objective hyperparameter optimization with optuna

To further improve model performance following transfer learning, a multi-objective HPO was performed using the Optuna framework. This study does not propose a new optimization algorithm but rather applies Optuna's existing multi-objective capabilities to the specific context of embedded, real-time FDM defect detection. Optuna [13] was selected for hyperparameter optimization because it provides an efficient, reproducible, and hardware-aware framework particularly suitable for embedded deep-learning applications. As compared to physics-based or grid-search methods, Optuna employs a Tree-structured Parzen Estimator (TPE) sampler and early pruning mechanism, enabling adaptive exploration of large parameter spaces and improve the likelihood of identifying high-performing configurations. Optuna provides a systematic and resource-efficient exploration of large parameter spaces. In addition, its pruning mechanism enables early termination of underperforming trials, saving computational resources. Combined with its support for multi-objective optimization, this allows simultaneous balancing of detection accuracy (mAP@0.5), localization precision (mAP@0.5:0.95) and inference speed. This makes it ideal for tuning real-time defect-detection models where performance depends on both accuracy and inference latency. The selection of Optuna is consistent with recent advancements in AM studies on data-driven process optimization in additive manufacturing, where

Table 3 Hyperparameter search space used in Optuna-based multi-objective optimization for YOLOv11n and YOLOv12n models

Hyperparameter	Type	Search Range / Values	Description
lr0	Continuous (log)	1e-5 to 1e-2	Initial learning rate
Lrf	Continuous	0.01 to 1.0	Final learning rate factor
weight_decay	Continuous (log)	1e-6 to 1e-2	Weight decay coefficient for regularization
warmup_epochs	Integer	0 to 5	Number of warm-up epochs
Scale	Continuous	0.0 to 0.9	Label scaling factor (for bounding box targets)
Dropout	Continuous	0.0 to 0.5	Dropout probability during training
Batch	Categorical	{8, 16, 32, 64}	Batch size used for training
Epochs	Integer	50 to 100	Maximum training epochs per trial
Patience	Integer	10 to 20	Early stopping patience (in epochs)

learning-based approaches have successfully complemented physics-based models [15, 16].

In this study, the optimization aimed to jointly maximize two critical performance metrics: mAP@0.5 (mean Average Precision at IoU=0.5) and mAP@0.5:0.95 (mean Average Precision averaged across IoU thresholds from 0.5 to 0.95). These metrics make sure that both general detection and precise localization are optimized, which are essential for real-time FDM defect monitoring. The complete search space, including parameter types, value ranges, and training roles, is summarized in Table 3, providing a comprehensive overview of the variables considered during Optuna-based optimization. A total of nine hyperparameters were selected for tuning across 50 trials per model. All Optuna-based hyperparameter optimization was executed on Google Colab Pro with an A100 GPU for parallel trial evaluation and early stopping.

Each trial involved sampling a distinct combination of hyperparameters from the defined search space, followed by fine-tuning the model (initialized with COCO-pretrained weights) on the 3D printing defect dataset. Model performance was then assessed using the validation set. To improve computational efficiency, Optuna's pruning mechanism was employed to halt underperforming trials at an early stage. All experiments were conducted on an NVIDIA A100 GPU, with trial metadata systematically logged in a persistent SQLite database. A detailed breakdown of the optimization workflow is provided in Table 4, which summarizes the key stages of sampling, training, pruning, and

Table 4 Multi-objective optimization workflow using Optuna for jointly maximizing mAP@0.5 and mAP@0.5:0.95. The algorithm integrates TPE sampling, early pruning, and Pareto front analysis to identify optimal YOLO hyperparameter configurations

Algorithm 2: Multi-Objective Hyperparameter Optimization using Optuna

Input: Pretrained YOLO model, defined hyperparameter search space

Output: Pareto-optimal hyperparameter configurations maximizing detection accuracy and localization precision

1. Define objective functions: maximize mAP@0.5 and mAP@0.5:0.95
2. Initialize Optuna study with TPE sampler and SQLite backend
3. For each trial t in total trials:
 - a. Sample a hyperparameter set from the search space
 - b. Load pretrained model weights (COCO)
 - c. Apply sampled parameters (e.g., lr0, batch, dropout)
 - d. Train model using the AdamW optimizer for up to 100 epochs
 - e. Evaluate validation mAP scores
 - f. Prune the trial if intermediate metrics fall below the threshold
4. Store results in the Optuna study database
5. Construct the Pareto front from non-dominated solutions
6. Select final model configurations based on balanced trade-offs

Pareto-based selection used to identify the most effective hyperparameter configurations.

The optimization process generated a set of Pareto-optimal configurations, each reflecting a balanced trade-off between detection accuracy and localization precision. The top-performing trials were selected for final evaluation and deployment. In comparison to the baseline (default) settings, the models tuned via Optuna demonstrated substantial improvements in both mAP@0.5 and mAP@0.5:0.95 scores, underscoring the effectiveness of a structured, multi-objective optimization approach for enhancing defect detection in FDM 3D printing.

3.5 Training configuration

The training setup for YOLOv11n and YOLOv12n models was organized into two key phases: transfer learning and HPO. In the initial transfer learning phase, both models were fine-tuned using the Stochastic Gradient Descent (SGD) optimizer, chosen for its stability and ability to preserve generalizable visual features inherited from COCO-pretrained weights. To support smooth convergence and

prevent overfitting, a cosine annealing learning rate scheduler was employed, gradually reducing the learning rate throughout the training.

On the other hand, during the Optuna-based HPO trials, the optimizer was switched to AdamW, an adaptive optimizer that decouples weight decay from the learning rate. This choice facilitated more efficient navigation of the loss landscape and accelerated convergence during the optimization process. Optuna dynamically tuned optimizer-related parameters such as learning rate (lr0), weight decay, momentum, and dropout over a total of 50 trials per model. Throughout both training phases, input images were resized to 640×640 pixels, preserving spatial integrity while ensuring compatibility with YOLO model requirements. Each model was trained for a maximum of 100 epochs, with early stopping applied to prevent overfitting in cases where the validation loss plateaued. The patience parameter for early stopping was optimized during the HPO phase.

Loss functions were carefully selected to address the unique challenges of FDM defect detection. For classification, Focal Loss was used to address class imbalance by focusing the model's learning on difficult or under-represented defect classes. For bounding box regression, the Complete Intersection over Union (CIoU) loss was employed, which refines localization accuracy by accounting for overlap area, centre point distance, and aspect ratio alignment. The choice to switch from SGD to AdamW during the Optuna phase was driven by the need for faster convergence and more effective exploration of the loss landscape. While SGD offered stable gradient updates suitable for transfer learning, AdamW's adaptive update mechanism proved more effective for overseeing a wider range of hyperparameter configurations during multi-objective tuning.

All training was conducted on an NVIDIA A100 GPU (40 GB), which allowed for large batch sizes, accelerated training cycles, and efficient resource utilization. Model checkpoints were saved in PyTorch format after each trial. The best-performing models, determined by validation mAP scores, were selected for deployment and further evaluation in the embedded system experiments. An overview of the end-to-end pipeline is shown in Fig. 4, spanning dataset development, transfer-learning-based training, Optuna-based multi-objective HPO with TPE and pruning,

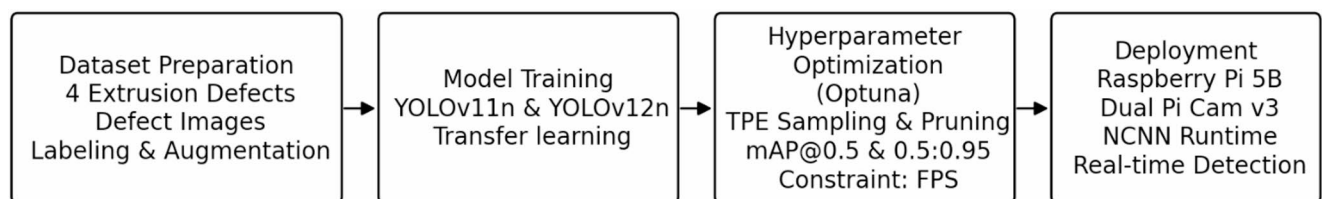


Fig. 4 Flow diagram of the proposed pipeline from dataset preparation and training through Optuna-based optimization to embedded deployment

and embedded deployment on a Raspberry Pi 5B with dual cameras.

3.6 Embedded deployment and real-time inference setup

To facilitate real-time defect detection within a resource-constrained embedded environment, the optimized YOLOv11n and YOLOv12n models were deployed on a Raspberry Pi 5B with 8 GB RAM. The system was equipped with two Raspberry Pi Camera Module 3 (IMX708) units, connected via CSI interfaces and configured to capture the front and side views of an Ender-3 V3 SE FDM printer. As shown in Fig. 5, the cameras were securely mounted using custom-designed 3D-printed brackets and articulated support arms, providing stable, vibration-resistant positioning throughout the print process. The camera assembly moves in coordination with the printer's Z-axis, allowing continuous monitoring of each successive layer during active printing. PLA filament was selected for all experiments due to its widespread use in FDM printing and its vulnerability

to extrusion-related defects, particularly stringing, under-extrusion, and over-extrusion, making it a representative material for evaluating the effectiveness of real-time defect detection systems.

The decision to use a dual-camera setup was determined by the need for multi-angle visual coverage, which improves the likelihood of detecting extrusion-related defects like stringing or spaghetti that may only be partially visible from a single viewpoint. In this setup, the side-facing camera was aligned parallel to the nozzle's motion within the X–Y plane, allowing horizontal tracking of filament deposition. The front-facing camera was oriented orthogonally to the print bed, enabling vertical monitoring of layer formation. This setup improved the ability of the system to detect defects reliably under different part geometries and lighting conditions and helped to maintain consistent tracking of how defects developed over time during the print.

The monitoring system was developed in Python using the *picamera2* library to support synchronized dual-camera streaming. The inference was performed using YOLO models exported in PyTorch (PT), Open Neural Network

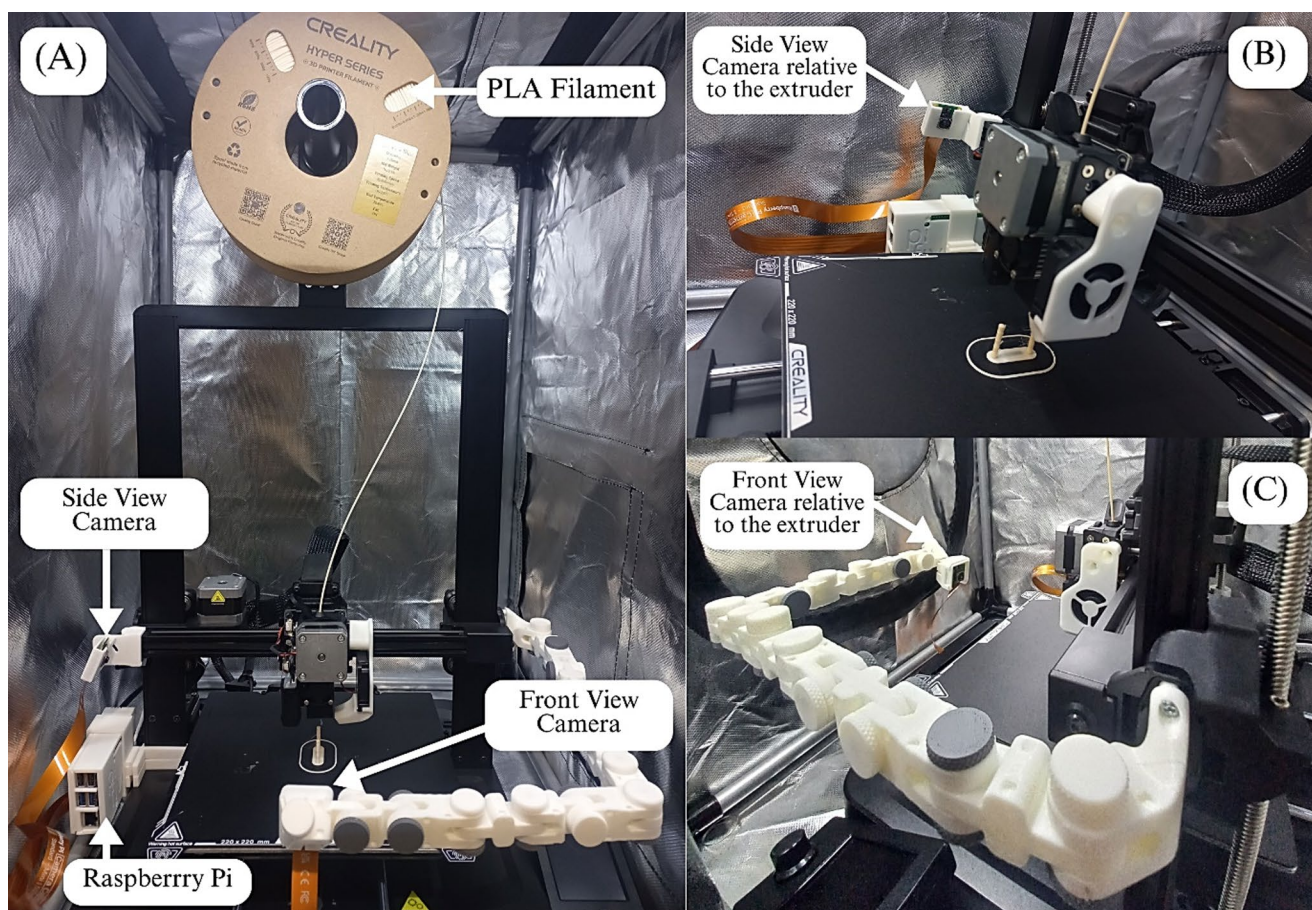


Fig. 5 Dual-camera setup for real-time defect detection **A** Full front view of the Ender-3 V3 SE printer with Raspberry Pi 5B, dual camera setup, and PLA filament. **B** Side view illustrating the relative position

of the side-facing camera to the extruder during printing. **C** Front-facing camera aligned with the nozzle for print monitoring

Exchange (ONNX), and Neural Network Computing Network (NCNN) formats to ensure compatibility across different deployment environments. A single script handled model loading, frame acquisition at 640×640 resolution, real-time inference, and overlaying of annotated detections. In addition to displaying results live, the script recorded video output and saved still images at fixed intervals. Unlike other real-time monitoring systems, this setup does not currently include G-code-triggered snapshot logic, relying instead on time-based capture intervals.

During the implementation, we observed and addressed different practical challenges. Early testing revealed a slight frame offset between the two cameras. This was mitigated by configuring identical frame rates and ensuring frame capture started only after a warm-up delay. As the system operates under ambient lighting, defects like stringing are hard to detect under shadows or reflections. The camera configuration was adjusted to minimize glare, and post-capture image brightness tuning is planned for future improvements.

The complete monitoring and recording pipeline ran entirely on the Raspberry Pi CPU within a virtual environment, demonstrating lightweight, reproducible deployment suitable for edge-based applications. All outputs, including frame-wise FPS, average inference latency, and file size, were logged automatically in structured formats to support benchmarking across model versions and formats. This deployment configuration forms the basis for future integration of real-time feedback into the printer controller for closed-loop defect correction.

4 Results

This section outlines the results from evaluating the YOLOv11n and YOLOv12n models on the FDM 3D printing defect detection. Both fine-tuned and Optuna-optimized models were evaluated to understand their performance in terms of accuracy, detection quality, and real-time capability. The results include comparisons between baseline and optimized models, analysis of each defect class, visualization of the optimization process using Pareto fronts, and real-time deployment outcomes on a Raspberry Pi 5B using a dual-camera setup. These findings show how the models perform across different conditions and highlight the improvements gained through optimization and deployment-specific adjustments.

4.1 Evaluation metrics

The performance of the proposed defect detection models was assessed using a set of metrics that capture both detection quality and suitability for real-time deployment:

- *mAP@0.5*: Mean Average Precision at an IoU threshold of 0.5. This reflects how well the model detects and classifies defects with moderate localization requirements. In practice, this metric emphasizes whether defects are correctly detected and classified, making it a useful indicator of detection accuracy.
- *mAP@0.5:0.95*: Average precision calculated across IoU thresholds from 0.5 to 0.95 (in 0.05 increments), providing a more comprehensive measure of detection and localization accuracy. This stricter metric penalizes imprecise bounding boxes, ensuring that the model not only detects a defect but also localizes it accurately. Together with mAP@0.5, it provides a balance between detection accuracy and localization precision.
- *Class-wise AP*: The average precision computed separately for each defect class: stringing, spaghetti, under-extrusion, and over-extrusion, offering detailed insight into per-class performance.
- *Confusion Matrix*: Used to visualize correct and incorrect predictions for each class, helping to analyse model behaviour and common misclassifications.
- *Inference Speed (FPS)*: The number of frames processed per second during live video inference on the Raspberry Pi 5B, indicating real-time performance capability.
- *Model Size (MB)*: The storage size of the exported model files in different formats (pt, onnx, and ncnn), relevant for deployment on memory-constrained devices.

Together, these metrics provide a comprehensive assessment of detection accuracy, localization robustness, and real-time performance critical for closed-loop FDM quality assurance systems.

4.2 Quantitative performance

This section compares the performance of fine-tuned and Optuna-optimized versions of YOLOv11n and YOLOv12n on the FDM defect detection task. Both models were evaluated across two phases: transfer learning using default configurations and multi-objective HPO using Optuna. The results, presented in Table 5, include mAP@0.5, mAP@0.5:0.95, and training configurations such as batch size, number of epochs, patience, and optimizer type. The corresponding hyperparameter values for the best-performing optimization trials are provided in Table 6.

The YOLOv11n-Finetuned model achieved mAP@0.5 of 81.66% and mAP@0.5:0.95 of 50.98% using the SGD optimizer with 91 epochs and a batch size of 32. After applying Optuna-based hyperparameter tuning, the best-performing YOLOv11n-Optimized (Best Trial 1) showed a moderate improvement to 83.11% mAP@0.5 and 51.06% mAP@0.5:0.95, using the AdamW optimizer. The

Table 5 Performance comparison of baseline (fine-tuned) and Optuna-optimized YOLOv11n and YOLOv12n models based on mAP@0.5, mAP@0.5:0.95, and training configuration parameters

Model	mAP@0.5 (%)	mAP@0.5:0.95 (%)	Batch Size	Epochs	Patience	Optimizer
YOLOv11n-Finetuned	81.66	50.98	32	91	10	SGD
YOLOv11n-Optimized (Best Trial 1)	83.11	51.06	32	97	20	AdamW
YOLOv11n-Optimized (Best Trial 2)	82.89	51.14	64	90	11	AdamW
YOLOv12n-Finetuned	81.61	51.32	32	100	10	SGD
YOLOv12n-Optimized (Best Trial 1)	84.28	52.40	16	96	15	AdamW
YOLOv12n-Optimized (Best Trial 2)	83.69	51.83	32	92	14	AdamW

YOLOv11n-Optimized (Best Trial 2) achieved 82.89% and 51.14%, respectively, indicating slightly better localization but slightly lower detection accuracy. YOLOv12n showed a more significant improvement through optimization. The baseline YOLOv12n-Finetuned achieved mAP@0.5=81.61% and mAP@0.5:0.95=51.32%. After optimization, YOLOv12n-Optimized (Best Trial 1) recorded a notable jump in performance with mAP@0.5=84.28% and mAP@0.5:0.95=52.40%. The YOLOv12n-Optimized (Best Trial 2) also outperformed the baseline, achieving 83.69% and 51.83% for mAP@0.5 and mAP@0.5:0.95, respectively.

These results highlight two main observations. First, YOLOv12n consistently gained more from HPO than YOLOv11n, with noticeable improvements in both detection and localization metrics. This can be attributed to architectural enhancements such as Area Attention and Flash Attention, which help capture fine-grained features and improve multi-scale detection. Second, YOLOv11n showed only marginal improvements, suggesting it may be more limited in its capacity to benefit from tuning. While still effective, its performance plateaued sooner than YOLOv12n, particularly in localization precision. Overall, the results confirm that combining a strong architecture like YOLOv12n with targeted optimization can significantly enhance performance, making it more suitable for real-time FDM defect detection in embedded environments.

4.3 Class-wise detection results

To further understand the strengths and limitations of each model, we evaluated the class-wise Average Precision (AP@0.5) for the four extrusion-related defect types: stringing, spaghetti, under-extrusion, and over-extrusion. This analysis was performed using the best hyperparameter-optimized versions of YOLOv11n and YOLOv12n. The results are summarized in Table 7, while Fig. 6A and B present the corresponding confusion matrices of optimized YOLOv11n and YOLOv12n, respectively, highlighting the classification behaviour across defect types.

Both models performed well on over-extrusion and under-extrusion, with YOLOv12n achieving 97.1% and 94.0%, and YOLOv11n close behind at 96.5% and 92.0%.

These results reflect the models' ability to detect clear, large-scale defects. For spaghetti, detections were approximately identical: 87.1% for YOLOv11n and 86.7% for YOLOv12n, indicating both models reliably identified this chaotic, high-contrast defect. The lowest scores were observed for stringing, with YOLOv12n slightly outperforming YOLOv11n (56.5% vs. 56.1%).

This performance trend is further explained by the relatively low AP@0.5 for stringing, which is attributed to its subtle and filament-thin nature that often blends with background textures and shadows, particularly in low-contrast conditions or when captured using wide field-of-view lenses. Unlike under- or over-extrusion, stringing lacks clear structural boundaries, making it more susceptible to false negatives or misclassification. Moreover, the fine-thread-like features may be underrepresented in the annotated dataset due to labelling ambiguity or image resolution limitations. To address this challenge, future work will involve capturing higher-resolution images focused specifically on stringing, incorporating multi-angle close-up views, and retraining the model using improved data augmentation techniques such as contrast boosting and edge sharpening. Integration of attention-refinement mechanisms or depth-assisted hybrid inputs may also enhance the model's ability to distinguish fine-grained defect patterns such as stringing. Overall, YOLOv12n showed marginally better performance across all classes, due to its attention-based enhancements. However, the persistent challenge in stringing detection suggests future work may require higher-resolution input or dataset refinement.

4.4 Optuna optimization outcomes and pareto front analysis

To explore the trade-off between detection accuracy and localization precision, multi-objective HPO was performed using Optuna for both YOLOv11n and YOLOv12n. Each model underwent 50 optimization trials aimed at jointly maximizing mAP@0.5 and mAP@0.5:0.95. The search space included nine hyperparameters (explained in Sect. 3.4), and Optuna's pruning strategy eliminated underperforming trials early to reduce compute time. The resulting optimization

Table 6 Selected hyperparameter values from the best Optuna trials for YOLOv11n and YOLOv12n, showing learning rate settings, label scaling, and regularization values

Model	Lr0	Lrf	Weight Decay	Warmup Epochs	Scale	Dropout
YOLOv11n-Optimized (Best Trial 1)	0.0048370864149788455	0.6024610010232	9.908789931506721e-05	2	0.3995617163514009	0.38929459166198405
YOLOv11n-Optimized (Best Trial 2)	0.0067156228037268196	0.1241916454707697	1.0200896839455274e-05	5	0.23931807316447665	0.051440616401771944
YOLOv12n-Optimized (Best Trial 1)	0.002139236677639142	0.8487226177122433	0.006909957345017695	2	0.0382048338523245	0.28837390595095286
YOLOv12n-Optimized (Best Trial 2)	0.002014066454108435	0.9136864166566925	0.000535855696360754	3	0.4654582781790004	0.40063079859671547

Table 7 Class-wise Average Precision (AP@0.5) for the best-performing Optuna-optimized versions of YOLOv11n and YOLOv12n on the test set

Defect Class	YOLOv11n-Optimized (Best Trial 1) (%)	YOLOv12n-Optimized (Best Trial 1) (%)
Stringing	56.1	56.5
Spaghetti	87.1	86.7
Under Extrusion	92.0	94.0
Over Extrusion	96.5	97.1

outcomes were visualized using Pareto fronts, shown in Fig. 7A and B for YOLOv11n and YOLOv12n, respectively.

As shown in Fig. 7A, YOLOv11n's Pareto front was relatively narrow, indicating a limited spread in localization precision ($\text{mAP}@0.5:0.95$). This suggests that while YOLOv11n can improve detection accuracy ($\text{mAP}@0.5$), its ability to localize defects more precisely plateaus quickly. In contrast, YOLOv12n's Pareto front in Fig. 7B was broader and shifted upward, indicating stronger optimization potential across both objectives. This is consistent with the results in Sect. 4.2, where YOLOv12n achieved higher mAP scores after optimization. For deployment, the top trials from each Pareto front were selected based on their balance between detection and localization performance. These results confirm the advantage of using multi-objective optimization when both metrics are critical for effective real-time defect detection in FDM 3D printing.

4.5 Real-time inference on embedded hardware

To evaluate practical deployment performance, the best-performing fine-tuned and Optuna-optimized versions of YOLOv11n and YOLOv12n were deployed on a Raspberry Pi 5B (8 GB RAM) connected to two Raspberry Pi Camera Module 3 units. These were mounted to capture front and side views of an Ender-3 V3 SE printer during active FDM printing. Each model was exported in three formats: PyTorch (.pt), ONNX (.onnx), and NCNN (.ncnn), and then evaluated for runtime performance on live dual-camera video feeds. The input resolution during inference was set to 640×640 , consistent with the live stream configuration used in the deployment script. The complete results are shown in Table 8, which summarizes model size (MB), average inference speed (FPS), and latency (ms) across all configurations.

Inference was performed using the CPU-only pipeline in a Python virtual environment without GPU acceleration. Among the tested formats, NCNN consistently delivered the best performance, with all models achieving over 50 FPS and average inference times below 20 ms, confirming its suitability for real-time applications. YOLOv11n-NCNN achieved the highest speed, reaching 57.31 FPS, while

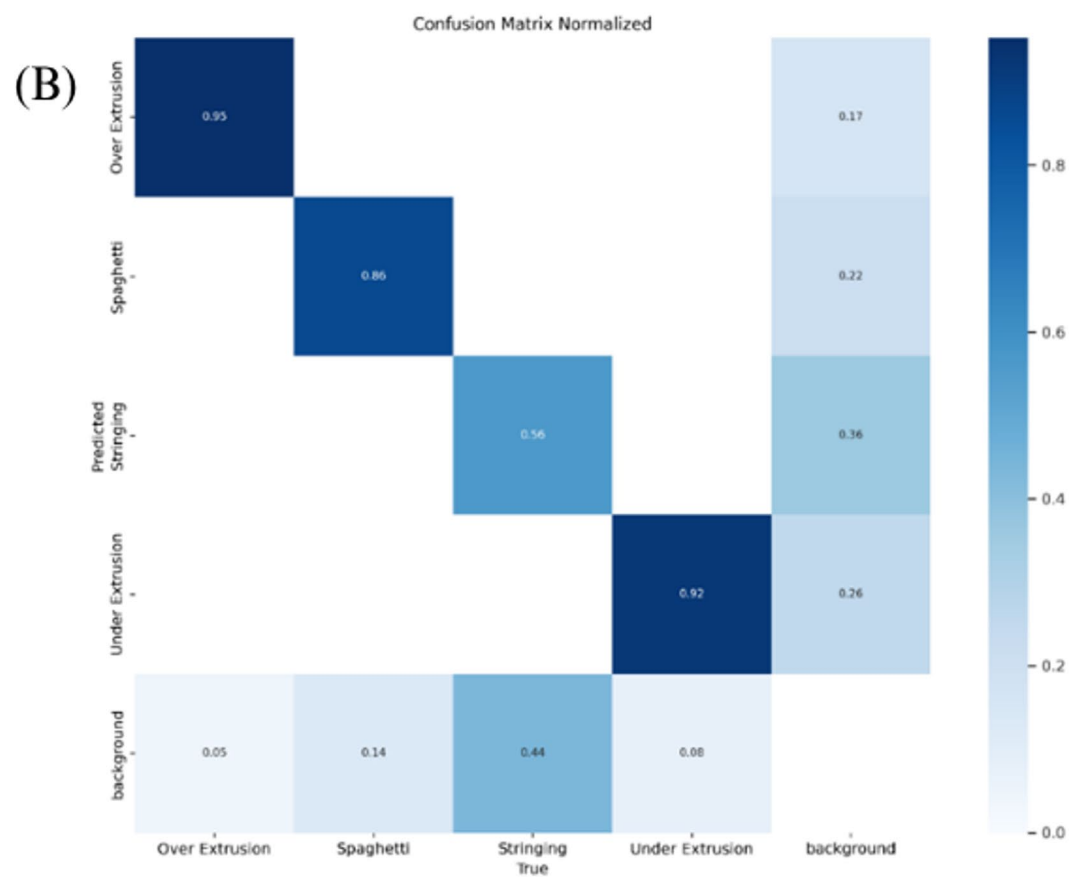
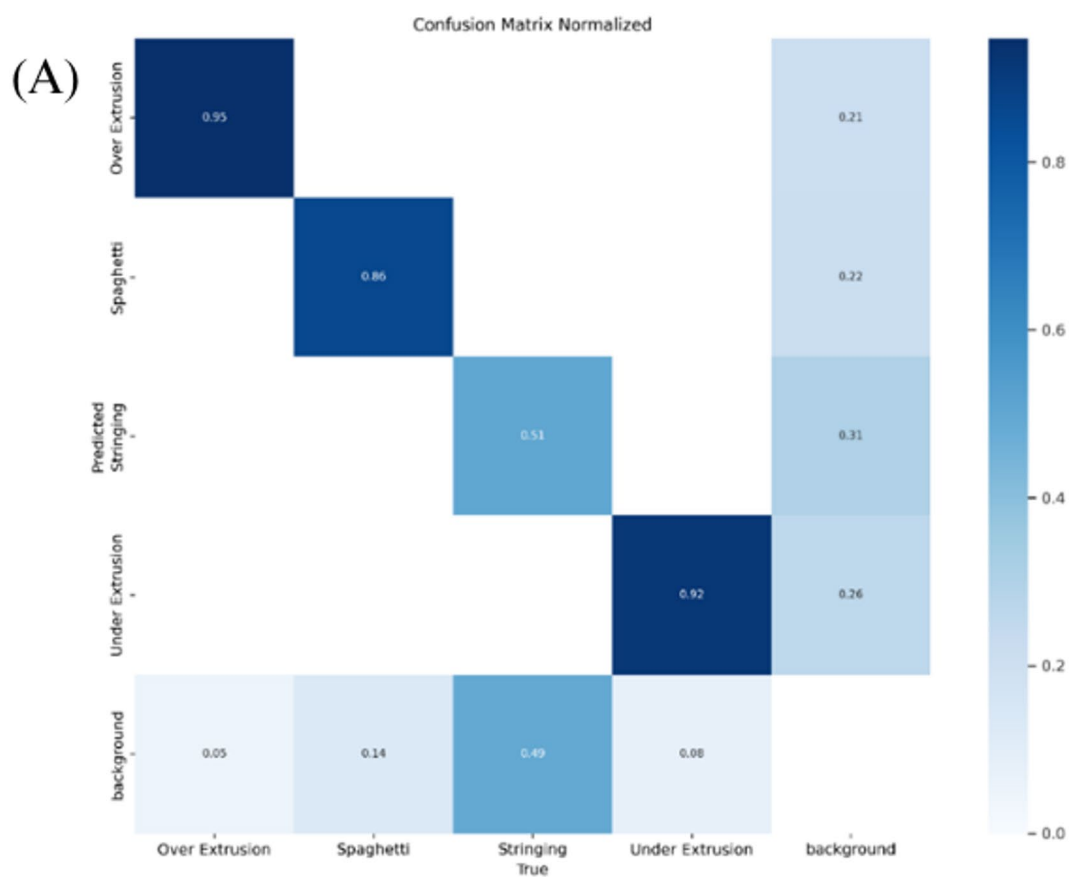


Fig. 6 Confusion matrix for the best optimized YOLO model showing classification results for the four extrusion-related defect types **A** YOLOv11n-Optimized (Best Trial 1), and **B** YOLOv11n-Optimized (Best Trial 1)

YOLOv12n-NCNN followed closely at 57.07 FPS. This performance advantage is primarily due to NCNN's lightweight inference engine and quantization-friendly structure.

ONNX models achieved moderate performance, typically ranging from 2.0 to 2.5 FPS with latency under 500 ms. These results suggest ONNX may be a viable fallback option for deployment scenarios where NCNN is unsupported. In contrast, PyTorch models showed the slowest performance, with average speeds between 1.2 and 1.5 FPS and inference times exceeding 800 ms, making them unsuitable for real-time deployment on the Raspberry Pi.

Moreover, while YOLOv12n models delivered better accuracy overall, YOLOv11n models outperformed them in terms of runtime speed across all deployment formats. This reflects the more compact architecture of YOLOv11n, which favours low-latency inference in constrained environments. These findings reinforce the importance of format selection and model-lightweighting for real-time embedded deployment. Overall, the NCNN model consistently achieved the highest FPS and lowest inference time, highlighting its efficiency and real-time suitability on the Raspberry Pi 5B. These results were recorded using the benchmark script described in Sect. 3.6, which streamed and saved performance data directly from the Raspberry Pi 5B using the dual Raspberry Pi camera module 3. Figure 8 illustrates real-time detection results using the YOLOv12n-Optimized (Best Trial 1) model on Raspberry Pi, showing accurate inference of various extrusion defects from dual camera views.

To complement the quantitative results presented above, four supplementary videos have been included to demonstrate real-time defect detection using the optimized YOLOv12n model on a Raspberry Pi 5B with dual-camera input. Real-time detection of stringing defects (Supplementary Video 1), real-time detection of spaghetti defects (Supplementary Video 2), real-time detection of under-extrusion defects (Supplementary Video 3), and real-time detection of over-extrusion defects (Supplementary Video 4). Each video showcases live inference using the optimized YOLOv12n model, capturing simultaneous front and side views of an actively printing object. It visually confirms the ability of the system to detect extrusion-related defects (e.g., stringing, spaghetti, under-extrusion, and over-extrusion) in real-time under typical ambient lighting conditions. This serves as a qualitative validation of the proposed pipeline and supports its practical deployment for intelligent defect monitoring in FDM 3D printing.

4.6 Comparative benchmarking against existing literature

To evaluate the novelty and effectiveness of the proposed real-time defect detection framework, we conducted a comprehensive comparison with recent studies that applied YOLO models to defect detection in FDM 3D printing. Table 9 summarizes key contributions from the literature, highlighting model types, defect categories handled, reported mAP@0.5 performance, and whether the solutions were deployed on embedded systems.

Cao et al. [9] implemented a YOLOv8-based model for extrusion-related defects achieving mAP@0.5 scores exceeding 90%. However, their solution was validated only on a high-end desktop GPU and lacked considerations for deployment on constrained hardware. Shabani et al. [23] integrated YOLO models with extrusion width monitoring, achieving dimensional accuracy within 90 μm . Although valuable for quality assessment, their system did not address multiple defect types or real-time embedded deployment. In another study, Ghate et al. [30] compared YOLOv8n and YOLOv11 for agricultural grading, with YOLOv8n achieving 98.25% mAP@0.5. However, their findings also emphasized the performance drop in YOLOv11 when not optimally tuned, highlighting the importance of hyperparameter optimization for model efficacy. Meanwhile, Hu et al. [34] demonstrated real-time defect detection using a lightweight YOLOv8 variant on a Jetson Nano, achieving 97.5% mAP@0.5. Despite promising results, the system used a single camera view and lacked multi-objective model optimization.

Although our YOLOv11n and YOLOv12n models reported relatively lower mAP scores compared to some of these benchmarks, these differences are primarily attributed to variations in datasets, defect scope, and experimental conditions. Specifically, most of the compared studies used domain-specific datasets (e.g., agricultural products or single-dimension extrusion width estimation) that differ substantially from the complex, multi-defect dataset captured under real-world FDM 3D printing conditions in our work. Furthermore, several studies did not evaluate their models on embedded hardware, thereby limiting their practical relevance.

On the other hand, this study focused on optimizing lightweight models for real-time embedded deployment on a Raspberry Pi 5B using a dual-camera setup. Each model was independently fine-tuned on our dataset using Optuna-based multi-objective hyperparameter optimization, rather than relying on pretrained weights or configurations from unrelated domains. This ensures fair comparison and demonstrates suitability for practical AM environments. Moreover, our approach introduces several implementation-level

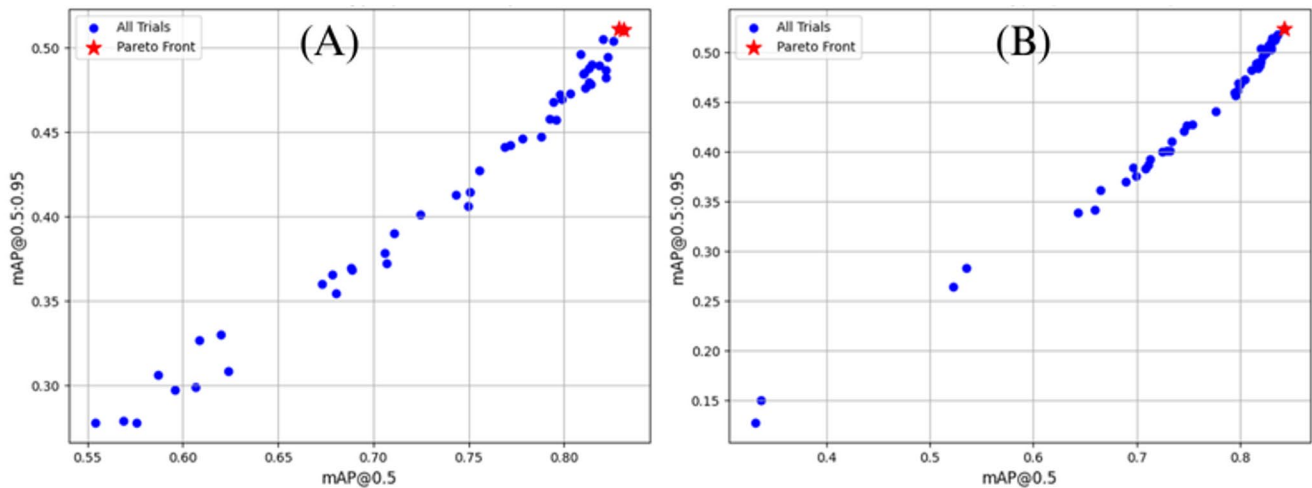


Fig. 7 Pareto front showing trade-offs between mAP@0.5 and mAP@0.5:0.95 across 50 Optuna optimization trials **A** Pareto front for YOLOv11n showing limited localization gains despite tuning. **B**

Table 8 Inference performance of fine-tuned and Optuna-optimized YOLOv11n and YOLOv12n models deployed on Raspberry Pi 5B, across three model formats

Model Name	Format	File Size (MB)	Avg FPS	Avg Inference Time (ms)
YOLOv11n-Finetuned	PT	5.22	1.51	671.4
YOLOv11n-Optimized (Best Trial 1)	PT	5.22	1.52	667.3
YOLOv11n-Optimized (Best Trial 2)	PT	5.22	1.50	671.7
YOLOv12n-Finetuned	PT	5.27	1.22	825.7
YOLOv12n-Optimized (Best Trial 1)	PT	5.27	1.16	876.0
YOLOv12n-Optimized (Best Trial 1)	PT	5.27	1.20	845.5
YOLOv11n-Finetuned	ONNX	10.10	2.39	429.9
YOLOv11n-Optimized (Best Trial 1)	ONNX	10.10	2.56	399.9
YOLOv11n-Optimized (Best Trial 2)	ONNX	10.10	2.56	400.9
YOLOv12n-Finetuned	ONNX	10.05	2.10	484.0
YOLOv12n-Optimized (Best Trial 1)	ONNX	10.05	2.12	478.3
YOLOv12n-Optimized (Best Trial 1)	ONNX	10.05	2.06	486.9
YOLOv11n-Finetuned	NCNN	10.01	57.31	18.3
YOLOv11n-Optimized (Best Trial 1)	NCNN	10.01	54.63	18.7
YOLOv11n-Optimized (Best Trial 2)	NCNN	10.01	52.44	19.7
YOLOv12n-Finetuned	NCNN	9.92	57.07	18.2
YOLOv12n-Optimized (Best Trial 1)	NCNN	9.92	54.72	18.8
YOLOv12n-Optimized (Best Trial 1)	NCNN	9.92	54.52	18.9

YOLOv12n exhibits a broader front, indicating better trade-offs and adaptability to multi-objective optimization

advancements. Specially, it presents a dual-camera detection pipeline on a Raspberry Pi 5B using the latest YOLOv11n and YOLOv12n models, optimized via Optuna for multi-objective performance. The best YOLOv12n configuration achieved mAP@0.5 = 84.28% and mAP@0.5:0.95 = 52.40% while maintaining inference speeds above 54 FPS in NCNN format, surpassing or closely approaching the accuracy of GPU-bound systems. Additionally, it uniquely targets four extrusion-specific defect types, offering broader and finer defect classification capabilities than existing works. These results demonstrate that integrating high detection precision, embedded feasibility, and multi-angle vision through structured optimization establishes a new practical benchmark for real-time AM quality assurance.

5 Discussions

This study investigated the use of YOLOv11n and YOLOv12n models for real-time detection of extrusion-related defects in FDM 3D printing. Both models were fine-tuned through transfer learning and further improved via Optuna-based multi-objective HPO. The final models were successfully deployed on a Raspberry Pi 5B with dual-camera input, demonstrating practical feasibility for low-cost, embedded defect monitoring. This setup achieved over 50 FPS using NCNN on a Raspberry Pi 5B, confirming that real-time defect detection is possible without cloud support or GPU acceleration.

While Table 9 includes studies reporting higher mAP scores, many of those results were obtained using different datasets, targeted limited defect categories, or lacked embedded system validation. On the other hand, our models

were optimized and evaluated specifically for real-time deployment on a Raspberry Pi 5B with dual-camera input, emphasizing practical feasibility over isolated benchmark scores. The core objective of this study was not merely to surpass other works in mAP, but to systematically improve the performance of lightweight YOLOv11n and YOLOv12n models (pretrained on the general-purpose COCO dataset) through Optuna-based multi-objective hyperparameter optimization configured for extrusion defect detection. This approach aligns with the need for adaptable, real-time monitoring solutions in constrained AM environments.

The results show that YOLOv12n consistently outperformed YOLOv11n across key detection metrics, particularly in mAP@0.5 and mAP@0.5:0.95. The optimized YOLOv12n also achieved slightly higher class-wise average precision on all four defect types, supported by its attention-driven architecture, which includes Area Attention and Flash Attention modules. These modules improve detection by capturing multi-scale spatial relationships and fine-grained features, especially useful for subtle extrusion anomalies like stringing and under-extrusion. These components improved the model's ability to focus on subtle and small-scale extrusion anomalies. While YOLOv11n demonstrated satisfactory performance, its improvements after optimization were more modest, suggesting architectural limitations in learning fine-grained features. However, optimized YOLOv11n retained advantages in runtime efficiency due to its lighter design, achieving higher FPS during NCNN-based inference. This speed advantage may be leveraged in feedback-based control pipelines, where rapid response is critical for implementing corrective print actions.

A key strength of this work is the successful deployment of both models on embedded hardware using CPU-only inference while maintaining real-time speeds. The optimized YOLOv12n model reached over 54 FPS using NCNN format on the Raspberry Pi 5B, validating the system's suitability for edge-based defect monitoring without requiring cloud support. In addition to accuracy and inference speed, energy consumption is an important factor for continuous industrial deployments. While this study primarily focused on balancing detection accuracy and runtime performance, it is noteworthy that lightweight models such as YOLOv11n and YOLOv12n inherently consume less power compared to larger architectures, particularly when deployed on CPU-only devices like the Raspberry Pi 5B. In our experiments, both models were able to operate at real-time speeds without external accelerators, highlighting their suitability for low-power, edge-based monitoring.

Beyond performance metrics, this work establishes a practical framework for real-time defect detection in AM. Specifically, the integration of Optuna-based multi-objective

optimization provides a systematic way to balance accuracy, localization precision, and inference speed under embedded hardware constraints. The creation of a dual-camera dataset capturing four extrusion defects from multiple perspectives further enhances robustness and offers a unique benchmark for the community. Importantly, this detection pipeline lays the groundwork for future closed-loop 3D printing, where defect identification can trigger corrective actions such as adjusting extrusion flow or pausing prints. Thus, the contribution extends beyond system implementation to advance the practical pathway toward autonomous and adaptive AM.

While the proposed framework demonstrates strong performance in polymer-based FDM, its applicability to other additive manufacturing processes should be interpreted carefully. The current system relies on RGB vision from dual cameras and is therefore suited specifically to visible extrusion defects. Extending the same YOLO-based pipeline to metal AM processes such as PBF or DED would require significant modifications, including the use of thermal, coaxial, or acoustic sensors and corresponding labeled datasets. Nevertheless, the underlying optimization and deployment pipeline, comprising Optuna-based multi-objective tuning, lightweight model design, and embedded inference, remains modular and can be adapted to other AM domains, provided appropriate sensing modalities and data are available.

Despite these overall improvements, stringing detection remained the most difficult task, with both models recording their lowest AP in this category. This is attributed to stringing's low contrast and fine visual features, which can be difficult to distinguish from minor artifacts or background noise. Addressing this challenge may require enhanced input resolution, improved lighting setups, or specialized detection heads.

In summary, this study demonstrates that combining architecture selection, transfer learning, and multi-objective optimization yields compact yet high-performing models for real-time 3D printing defect detection on embedded systems. Beyond reporting detection accuracy and speed, the work contributes a methodological framework tailored for resource-constrained AM environments and introduces a dual-camera dataset for robust evaluation across multiple extrusion defects. The optimized YOLOv12n emerges as the better all-around choice for accuracy, while YOLOv11n offers advantages in runtime efficiency, which may be valuable in feedback-based control pipelines. Together, these contributions provide not only practical feasibility but also a foundation for advancing toward defect-adaptive closed-loop 3D printing.

An important next step is the integration of detection outputs into corrective actions during printing. In a closed-loop implementation, defect classification and localization could serve as feedback signals to adjust process parameters in real

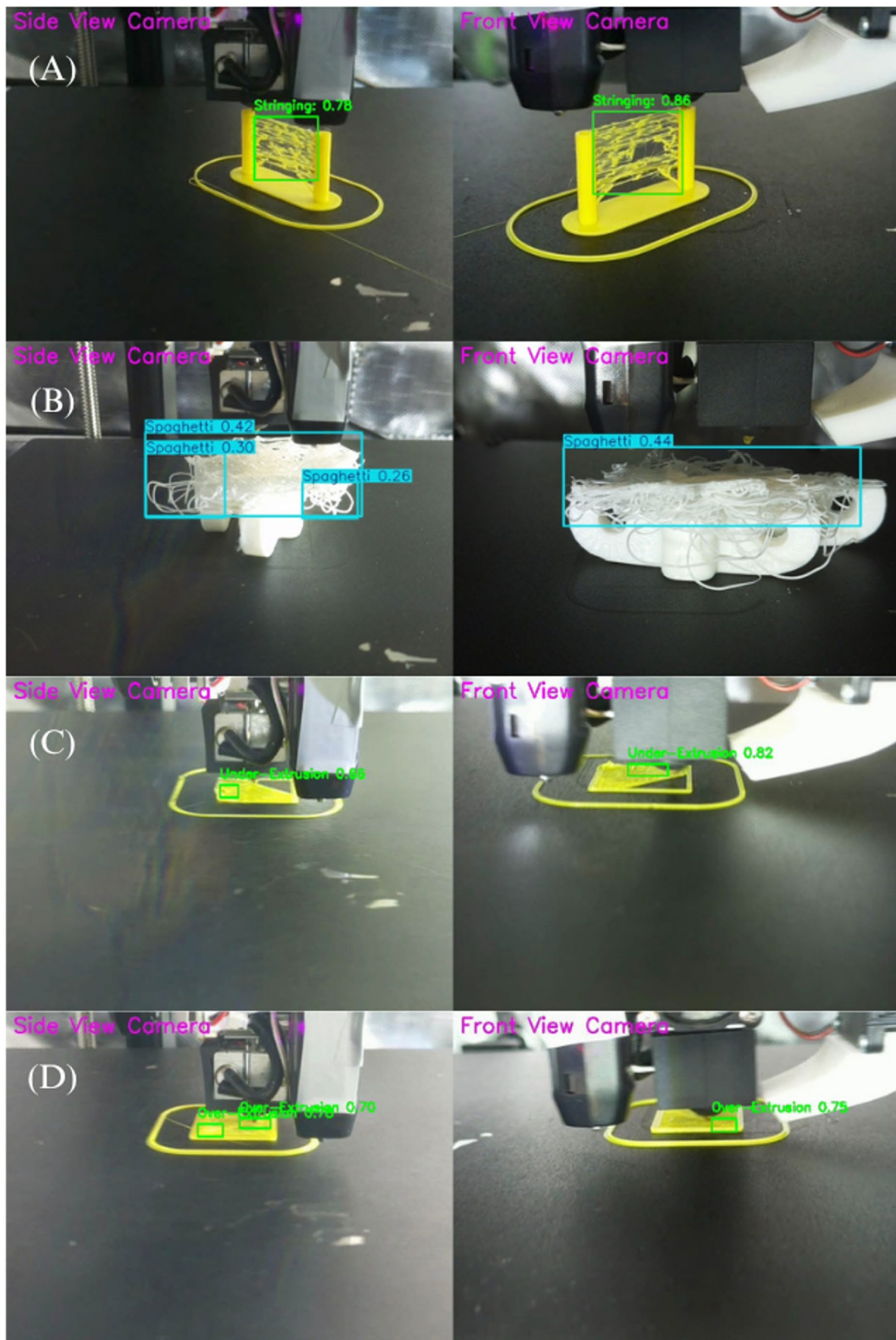


Fig. 8 Real-time detection of extrusion-related defects using the YOLOv11n-Optimized (Best Trial 1) model on the Raspberry Pi 5B. Each row shows side and front camera views of different defect types captured during active 3D printing. **A** Detection of stringing defects. **B** Detection of spaghetti defects. **C** Detection of under-extrusion defects. **D** Detection of over-extrusion defects

time. For instance, detecting under-extrusion could prompt an increase in flow rate, while over-extrusion detection might guide a reduction in material flow. Spaghetti formation could trigger an automatic pause of the print to prevent material waste, whereas stringing detection could inform temperature tuning or retraction adjustments. Although this study focused on validating the detection framework, these examples illustrate how the system can function as the perception layer of a closed-loop control pipeline, ultimately enabling adaptive and autonomous 3D printing.

5.1 Closed-loop system feasibility and latency analysis

Figure 9 presents the complete perception-to-action pipeline that underpins the proposed closed-loop defect-adaptive printing framework. The workflow consists of five sequential stages: (1) image acquisition from dual cameras, (2) real-time defect inference via optimized YOLOv12n/YOLOv11n models, (3) decision-logic evaluation in a lightweight Python node that interprets the defect class and severity, (4) command transmission through the OctoPrint REST API or direct serial interface, and (5) printer response executed by the Marlin firmware.

Based on experimental timing logs during real-time monitoring, the average total system latency was estimated at approximately 36–42 ms per cycle, comprising ~18 ms for model inference (NCNN on Raspberry Pi 5 B CPU), ~10 ms for decision and buffering, and ~8–14 ms for API or serial communication. This latency corresponds to a control-loop rate of ~24 Hz, which is sufficient for layer-by-layer defect response in extrusion printing.

A proof-of-concept trigger test was also conducted offline to verify communication integrity: when a spaghetti-type defect was detected in recorded video frames, the controller node automatically issued an M25 (Pause Print) command via OctoPrint, which successfully halted the printer. Although this preliminary test was performed in simulation, it demonstrates that detection outputs can directly initiate corrective actions through the same communication pathway. These results confirm that the proposed monitoring framework can function as the perception layer of an AI-augmented, closed-loop additive-manufacturing system.

Although this work focused on real-time detection and communication validation, a physics-informed control simulation will be developed in subsequent stages to evaluate

parameter adaptation (e.g., flow rate or temperature correction) within the same feedback framework. Future deployments can further reduce inference latency (from ~18 ms to below 10 ms) by integrating on-board AI accelerators such as the Raspberry Pi AI HAT (26 TOPS) or Google Coral USB, enabling faster perception action cycles for real-time corrective control.

5.2 Practical limitations and future works

While the proposed system demonstrated a robust performance in real-time defect detection, we also identified several limitations that could impact generalization and deployment in other settings. First, detection performance for stringing defects remained low, even with optimized models. Stringing typically presents as thin, low-contrast filaments that are visually subtle and easily confused with background noise. At the input resolution of 640×480 , stringing structures often span only 1 to 3 pixels in width, challenging the receptive field of convolutional layers and limiting discriminative learning. This makes it harder for the models to differentiate such patterns without additional context or enhanced input resolution.

Second, the models were trained and evaluated on a custom dataset derived from selected sources, with specific lighting conditions and camera angles. As a result, model performance may vary under different lighting setups, printer types, or camera configurations. In particular, glossy filaments, angled light reflections, and varying camera heights can significantly impact defect contrast and visibility. Additional data collection and domain adaptation techniques may be necessary to improve robustness across varying environments.

To further clarify robustness, we observed that detection accuracy can degrade under varying illumination, reflections from glossy filaments, and camera vibration induced by printer motion. Although augmentation techniques such as brightness and shear variation improved tolerance to some extent, environmental perturbations remain a potential source of false positives and missed detections. Future studies will systematically evaluate robustness by testing under different lighting setups, camera positions, and synthetic noise to better quantify the model's resilience in realistic operating conditions.

Third, while YOLOv12n delivered strong accuracy, it also had a larger computational footprint than YOLOv11n. Although both models ran in real-time using NCNN on the Raspberry Pi 5B, this balance between model complexity and speed may shift on older or less capable hardware. Power consumption and heat dissipation under continuous operation were not evaluated but should be considered in long-term deployments.

Table 9 Comparative Benchmarking of YOLO-Based Models for FDM Defect Detection

Study	YOLO Version	Defect Types	mAP@0.5	Embedded Deployment	Notes
[9]	YOLOv8	Under-extrusion, Over-extrusion	92.1%	No (Desktop GPU)	High accuracy but no embedded validation
[23]	YOLOv5	Extrusion width	N/A (Dimensional error ~90 µm)	No	Integrated width measurement, not multi-defect
[30]	YOLOv8n, YOLOv11	Arcanut grading	98.25 (YOLOv8n)	No	Focused on agricultural defects
[34]	YOLOv8-lite	Stringing, warping, Cracking	97.5	Yes (Jetson Nano)	Good real-time results, but single camera only
This study	YOLOv11n, YOLOv12n	Stringing, Spaghetti, Under/Over-extrusion	84.28% (YOLOv12n)	Yes (Raspberry Pi 5B)	Dual-camera setup; multi-objective optimization via Optuna

Another promising direction is to systematically evaluate the trade-off between detection accuracy and energy consumption in embedded devices. Since long-duration industrial printing requires power-efficient solutions, future studies will investigate model architectures, optimization

strategies, and hardware accelerators that minimize energy usage while maintaining high detection performance.

Finally, feedback integration into the printer's control system was outside the scope of this work. While real-time inference was achieved, this study did not implement closed-loop control to pause or adjust printing based on defect detection. Future work can explore full feedback-based corrective printing mechanisms.

Future work will focus on extending the current detection pipeline into a fully closed-loop 3D printing system capable of autonomous defect correction. While this study demonstrated accurate real-time detection, integrating model output with the printer's firmware or controller remains a key step. Future work could explore integrating detection outputs with OctoPrint APIs or Klipper macros to enable automatic G-code pauses, retractions, or flow adjustments based on detected defects. This would enable actions such as pausing the print, adjusting extrusion flow, or modifying temperature and speed in response to detected defects, effectively establishing a feedback loop between monitoring and process control.

Another priority is improving detection of stringing, the most difficult defect type in this study. Addressing this challenge may involve using higher-resolution input, improved lighting setups, or multi-view fusion, combining data from both top and front cameras. Additional preprocessing such as CLAHE (Contrast Limited Adaptive Histogram Equalization) and edge sharpening may enhance model sensitivity to filament-thin defects. Integrating depth sensing (e.g., stereo vision or LiDAR) could also improve model accuracy by providing 3D spatial cues for fine-grained extrusion anomalies.

Moreover, supporting online or continual learning could make the system more adaptable to changing materials, geometries, and environmental conditions. Lightweight continual learning strategies such as Elastic Weight Consolidation (EWC) or rehearsal-based memory buffers can be explored to maintain adaptability without full retraining. This would allow the model to self-update during operation, improving robustness over time. From a usability perspective, future systems may benefit from explainable AI

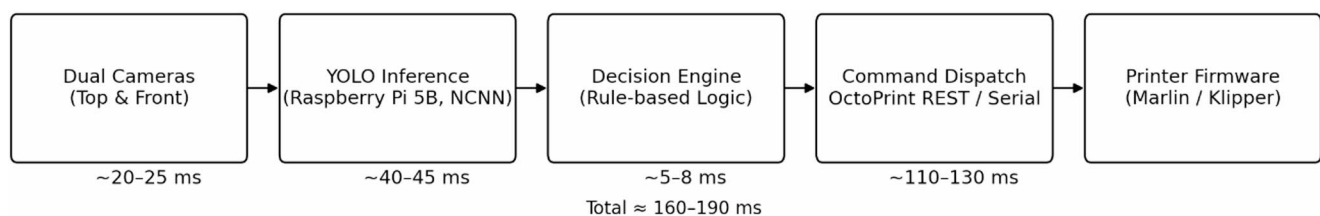


Figure: Camera → YOLO Inference → Decision Engine → G-code/Serial Command → Printer Response.
Example: spaghetti detection triggers M25 (Pause Print).

Fig. 9 Perception-to-action pipeline illustrating the closed-loop defect detection and control sequence in FDM 3D printing, showing latency from camera capture to printer response

(XAI) techniques, such as Grad-CAM or uncertainty visualization, to improve interpretability and operator trust, particularly important during early-stage industrial deployment. Together, these advancements will help realize a closed-loop, intelligent 3D printing ecosystem, combining AI-driven monitoring, real-time inference, and automated correction to ensure consistent print quality and reduce waste.

6 Conclusion

This study presented a comprehensive evaluation of YOLOv11n and YOLOv12n for real-time detection of extrusion-related defects in FDM 3D printing. A custom-labelled dataset covering four key defect types was developed and used to fine-tune both models using transfer learning. To improve detection performance, a multi-objective HPO approach was implemented using Optuna, jointly maximizing detection accuracy (mAP@0.5) and localization precision (mAP@0.5:0.95). Experimental results showed that optimized YOLOv12n consistently outperformed the optimized YOLOv11n in detection accuracy, localization precision, and class-wise average precision. The YOLOv12n's attention-based enhancements enabled more effective detection of subtle and small-scale extrusion anomalies. While the optimized YOLOv11n remained slightly faster during inference on embedded hardware, its detection performance plateaued, suggesting it is better suited for latency-critical applications.

Both models were successfully deployed on a Raspberry Pi 5B using NCNN, achieving real-time inference speeds above 50 FPS, confirming their viability for low-cost, embedded monitoring systems. Despite these achievements, stringing detection remained the most challenging task due to its fine-grained visual nature. The proposed pipeline, which combines model architecture selection, transfer learning, multi-objective optimization, and embedded deployment, lays the foundation for intelligent, real-time quality assurance in FDM 3D printing. These results demonstrate that lightweight, optimized deep learning models can be integrated into compact low-resource hardware for efficient defect detection.

Beyond comparative model performance, this study establishes a practical framework for real-time defect detection tailored to embedded AM systems. The integration of Optuna-based multi-objective optimization with a custom dual-camera dataset provides a systematic approach for balancing accuracy, localization precision, and inference speed under resource constraints. Demonstrating feasibility on a Raspberry Pi 5B highlights the potential for practical, low-cost deployment in industrial and research settings.

Importantly, this detection pipeline lays the groundwork for future closed-loop, defect-adaptive 3D printing, where real-time monitoring can be directly coupled with corrective actions to enable autonomous and adaptive manufacturing.

Future work will focus on extending this system into a closed-loop 3D printing and 4D printing environment, where real-time detection can trigger automated corrective actions such as pausing, reprinting, or adjusting process parameters. Such advancements will contribute toward fully autonomous, AI-augmented AM systems aligned with the goals of Industry 4.0.

Supplementary Information The online version contains supplementary material available at <https://doi.org/10.1007/s40964-025-01401-0>.

Acknowledgements Ali Zolfagharian acknowledges the support by Australian Research Council (ARC) under the Discovery Early Career Research Award (DECRA) [project number DE240100960] funded by the Australian Government.

Funding This research is supported by Australian Research Council under the Discovery Early Career Researcher Award (ARC DECRA) [project number DE240100960] funded by the Australian Government.

Data availability Datasets generated during the current study are available from the corresponding author on reasonable request.

References

- Halim NA et al (2021) A review on 3D printed polymer-based composite for thermal applications. IOP Conf Ser Mater Sci Eng 1078(1):012029. <https://doi.org/10.1088/1757-899X/1078/1/012029>
- Wickramasinghe S, Do T, Tran P (2020) FDM-based 3D printing of polymer and associated composite: a review on mechanical properties, defects and treatments. Polymers 12(7):1529. <https://doi.org/10.3390/POLYM12071529>
- Sani AR, Zolfagharian A, Kouzani AZ (2024) Automated defects detection in extrusion 3D printing using YOLO models. J Intell Manuf. <https://doi.org/10.1007/S10845-024-02543-8/TABLES/5>
- Avro SS, Rahman SMA, Tseng TL, Rahman MF (2024) A deep learning framework for automated anomaly detection and localization in fused filament fabrication. Manuf Lett 41:1526–1534. <https://doi.org/10.1016/J.MFGLET.2024.09.179>
- Bajrami A, Palpacelli MC, Lettera G, Pantanetti S (2024) A comparative analysis on a limited image dataset for accurately detecting improperly polished surfaces for industrial applications. In: MESA 2024—Proceedings of 20th international conference on mechatronic, embedded systems and applications. <https://doi.org/10.1109/MESA61532.2024.10704894>
- Yue Z, et al (2024) Real-time in-situ defect detection in 3D printing of continuous fiber-reinforced composites. In: Proceedings—2024 IEEE 4th international conference on digital twins and parallel intelligence, DTPI 2024, pp 541–546. <https://doi.org/10.1109/DTPI61353.2024.10778908>
- Khusheef AS, Shahbazi M, Hashemi R (2023) Investigation of long short-term memory networks for real-time process monitoring in fused deposition modeling. Progress Addit Manufact

- 8(5):977–995. <https://doi.org/10.1007/S40964-022-00371-X/FIGURES/18>
8. Ansari MA, Crampton A, Parkinson S (2024) Keyhole porosity identification and localization via X-Ray imaging with YOLO. *IEEE Access* 12:61049–61061. <https://doi.org/10.1109/ACCESS.2024.3393128>
9. Cao M, Lijun fu, Kui Zhou, Ai F (2024) Improved 3d printing extrusion defect detection method based on Yolo-V8, <https://doi.org/10.2139/SSRN.4800007>.
10. Rasheed AF, Zarkoosh M (2025) YOLOv11 optimization for efficient resource utilization. *J Supercomput* 81(9):1–21. <https://doi.org/10.1007/S11227-025-07520-3/FIGURES/13>
11. Ji Y et al (2025) Transmission line defect detection algorithm based on improved YOLOv12. *Electronics* 14(12):2432. <https://doi.org/10.3390/ELECTRONICS14122432>
12. Liu K (2024) Driving towards a greener future: the role of artificial intelligence and 3D printing in advancing energy-efficient vehicles, pp 688–703. https://doi.org/10.2991/978-94-6463-518-8_66
13. Akiba T, Sano S, Yanase T, Ohta T, Koyama M (2019) Optuna: a next-generation hyperparameter optimization framework. In: *Proceedings of the ACM SIGKDD international conference on knowledge discovery and data mining*, pp 2623–2631. <https://doi.org/10.1145/3292500.3330701>
14. Yang CJ, Huang WK, Lin KP (2023) Three-dimensional printing quality inspection based on transfer learning with convolutional neural networks. *Sensors (Basel)* 23(1):491. <https://doi.org/10.3390/S23010491>
15. Veeman D, Vellaisamy M, Ponnusamy PC, Subramaniyan MK, Vijayakumar MD, Guo L (2025) Influence of optimization techniques on machine learning algorithms: compressive behaviour of additively manufactured poly lactic acid (PLA) for structural applications. *Progress Addit Manuf* 10(4):2613–2630. <https://doi.org/10.1007/S40964-024-00770-2/FIGURES/13>
16. Veeman D, Vellaisamy M, Ponnusamy PC, Subathra DP, Katiyar JK (2025) Impact of process parameters on hardness of melt fabricated PA6 carbon fiber-reinforced composites and prediction of properties using machine learning. *J Mater Eng Perform*. <https://doi.org/10.1007/S11665-025-11406-Z/FIGURES/11>
17. Sani AR, Zolfagharian A, Kouzani AZ (2024) Artificial intelligence-augmented additive manufacturing: insights on closed-loop 3D printing. *Adv Intell Syst* 6(10):2400102. <https://doi.org/10.1002/AISY.202400102>
18. Siddiqui MMUZ, Tabassum A (2025) Condition-based monitoring techniques and algorithms in 3d printing and additive manufacturing: a state-of-the-art review. *Progress Addit Manuf* 10(8):4257–4304. <https://doi.org/10.1007/S40964-024-00816-5/FIGURES/14>
19. Hussain M (2023) YOLO-v1 to YOLO-v8, the rise of YOLO and its complementary nature toward digital manufacturing and industrial defect detection. *Machines* 11(7):677. <https://doi.org/10.3390/MACHINES11070677>
20. Deepti Raj G, Prabadevi B (2025) Enhancing surface detection: a comprehensive analysis of various YOLO models. *Heliyon*. <https://doi.org/10.1016/J.HELIYON.2025.E42433>
21. Nikam D, Chukwuemeke A, Nigam A, Bhosale T, Nikam S (2025) On the application of YOLO-based object detection models to classify and detect defects in laser-directed energy deposition process. *Progress Addit Manuf* 10(10):7609–7624. <https://doi.org/10.1007/S40964-025-01056-X/FIGURES/10>
22. Wang X, Gao H, Jia Z (2023) Li Z (2023) BL-YOLOv8: An Improved Road Defect Detection Model Based on YOLOv8. *Sensors* 23(20):8361. <https://doi.org/10.3390/S23208361>
23. Shabani A, Martinez-Hernandez U (2023) In-Situ measurement of extrusion width for fused filament fabrication process using vision and machine learning models. In: *IEEE international conference on intelligent robots and systems*, pp 8298–8303. <https://doi.org/10.1109/IROS55552.2023.10341406>.
24. Sapkota R et al (2025) YOLO advances to its genesis: a decadal and comprehensive review of the You Only Look Once (YOLO) series. *Artif Intell Rev* 58(9):1–83. <https://doi.org/10.1007/S10462-025-11253-3/TABLES/8>
25. Zhu W, Han X, Zhang K, Lin S, Jin J (2025) Application of YOLO11 model with spatial pyramid dilation convolution (SPD-Conv) and effective squeeze-excitation (EffectiveSE) fusion in rail track defect detection. *Sensors* 25(8):2371. <https://doi.org/10.3390/S25082371>
26. El Zeinaty C, Hamidouche W, Herrou G, Menard D (2025) Designing object detection models for TinyML: foundations, comparative analysis, challenges, and emerging solutions. *ACM Comput Surv*. <https://doi.org/10.1145/3744339>
27. Nguyen PT, Huynh DC, Ho LD, Dunnigan MW (2025) YOLO-WTB: improved YOLOv12n model for detecting small damage of wind turbine blades from aerial imagery. *IEEE Access*. <https://doi.org/10.1109/ACCESS.2025.3589225>
28. Bakir C, Gezer A (2025) Real-time automatic detection of nutrient deficiency in lettuce plants with new YOLOV12 model. *J Sens* 2025(1):5592225. <https://doi.org/10.1155/JS/5592225>
29. Ramos L, Casas E, Bendek E, Romero C, Rivas-Echeverría F (2024) Hyperparameter optimization of YOLOv8 for smoke and wildfire detection: implications for agricultural and environmental safety. *Artif Intell Agric* 12:109–126. <https://doi.org/10.1016/J.AIIA.2024.05.003>
30. Ghate D, Saishma H, Adithya M, Ghate SD (2025) Advancing arecanut quality grading: a comparative analysis of YOLO models with hyperparameter optimization. <https://doi.org/10.21203/R.S.3.RS-5755373/V1>.
31. Yosinski J, Clune J, Bengio Y, Lipson H (2025) Advances in neural information processing systems, 2014 proceedings.neurips.cc. Accessed 12 May 2025. Available: <https://proceedings.neurips.cc/paper/5347-how-transferable-are-features-in-deep-n%E2%80%A6>
32. Li H, Li X, Fan Q, Xiong Q, Wang X, Leung VCM (2024) Transfer learning for real-time surface defect detection with multi-access edge-cloud computing networks. *IEEE Trans Network Serv Manag* 21(1):310–323. <https://doi.org/10.1109/TNSM.2023.3301718>
33. Hendrycks D, Lee K, Mazeika M (2025) *Proceedings of international conference on machine learning, 2019 MLR press*, Accessed: May 12, 2025. <https://proceedings.mlr.press/v97/hendrycks19a.html>
34. Hu WJ, Chen C, Su S, Zhang J, Zhu A (2024) Real-time defect detection for FFF 3D printing using lightweight model deployment. *Int J Adv Manuf Technol* 134(9–10):4871–4885. <https://doi.org/10.1007/S00170-024-14452-4/FIGURES/13>
35. Ajay V, Shrivastava A (2025) A novel single-camera-based vision system for real-time measurement of bead width and height during wire arc additive manufacturing. *Progress Addit Manuf* 10(9):5923–5935. <https://doi.org/10.1007/S40964-024-00943-Z/FIGURES/12>
36. “3d-printing-failure-detection Dataset > Overview.” Accessed 13 May 2025. Available: <https://universe.roboflow.com/sylucauc/3d-printing-failure-detection>
37. Khan AT, Jensen SM (2025) Leaf-net: a unified framework for leaf extraction and analysis in multi-crop phenotyping using YOLOv11. *Agriculture (Basel)* 15(2):196. <https://doi.org/10.3390/AGRICULTURE15020196>
38. Lin TY, et al (2014) Microsoft COCO: common objects in context. In: *Lecture notes in computer science (including subseries lecture notes in artificial intelligence and lecture notes in Bioinformatics)*, vol 8693 LNCS, no. PART 5, pp 740–755, https://doi.org/10.1007/978-3-319-10602-1_48

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.